

Exame Modelo 1 — Paradigmas de Programação — Época de Recurso 2025/2026

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época de Recurso
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
- Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.

Parte 1

Pergunta 1 (1,5 valores)

Explique detalhadamente o conceito de encapsulamento em Programação Orientada a Objetos. Descreva como os modificadores de acesso (`private` , `protected` , `public`) contribuem para este princípio. Justifique por que razão o acesso direto aos atributos de uma classe é considerado uma má prática e apresente um exemplo prático que demonstre o uso correto de getters e setters com validação.

Pergunta 2 (1,5 valores)

Explique o conceito de polimorfismo em Java, distinguindo entre polimorfismo de sobrecarga (*overloading*) e polimorfismo de sobreposição (*overriding*). Descreva as regras que devem ser respeitadas em cada caso e forneça exemplos de código que ilustrem ambos os tipos de polimorfismo. Explique também como a JVM determina qual o método a invocar em tempo de execução no caso da sobreposição.

Pergunta 3 (1,5 valores)

Explique o papel da classe `Object` como raiz da hierarquia de classes em Java. Descreva os métodos mais importantes que todas as classes herdam de `Object` (`equals()`, `toString()`, `getClass()`) e justifique em que circunstâncias devem ser redefinidos. Ilustre com um exemplo prático que demonstre as consequências de não redefinir o método `equals()`.

Pergunta 4 (1,5 valores)

Explique o conceito de herança em Java. Descreva o papel da palavra reservada `super` e os diferentes contextos em que pode ser utilizada (construtor e métodos). Discuta as limitações da herança simples em Java e como as interfaces podem ser usadas para contornar essas limitações. Ilustre com um exemplo prático.

Parte 2

1. Considere as seguintes interfaces `AidBox` e `Container`. A interface `AidBox` descreve as operações possíveis que definem o contrato de uma caixa de suprimentos utilizada na recolha de bens de uma instituição de ajuda humanitária. A interface `Container` define o contrato de um contentor associado a uma `AidBox`.

```
public interface AidBox {  
    String getCode();  
    String getZone();  
    Container[] getContainers();  
    boolean equals(Object obj);  
}
```

```
public interface Container {  
    String getCode();  
    ItemType getType();  
    double getCapacity();  
    Measurement[] getMeasurements();  
    Measurement getLastMeasurement();  
}
```

```
public enum ItemType {  
    PERISHABLE_FOOD,  
    NON_PERISHABLE_FOOD,  
    CLOTHING,  
    MEDICINE  
}
```

```
public interface Measurement {  
    double getValue();  
    String getDate();  
}
```

Pergunta 1a (3 valores)

Considere a interface `AidBox` que representa uma caixa de suprimentos. Implemente a interface numa classe denominada `AidBoxImpl`. A `AidBox` deve possuir um array de `Container` com capacidade máxima de 4 contentores. Para a implementação do método `equals`, considere que **duas instâncias** de `AidBox` são iguais se possuírem o mesmo código (devolvido através do método `getCode()`). Implemente também um método `addContainer(Container container)` que adicione um contentor à `AidBox`, lançando uma exceção caso a capacidade máxima seja atingida.

Pergunta 1b (2 valores)

Desenvolva o código necessário para testar a classe implementada (por exemplo, no contexto de um método `main`). Apresente um exemplo de teste para cada método implementado.

1. Considere a existência de uma classe `ReportImpl` que implemente a interface `Report`.

```
public interface Report {  
    String generate(IInstitution inst);  
}
```

Pergunta 2a (4 valores)

Implemente os seguintes métodos que podem ser utilizados para a geração do relatório na classe

`ReportImpl`:

```
int countContainersByType(AidBox aidbox, ItemType type);
```

- Este método deve devolver o número de contentores existentes na `AidBox` cujo tipo seja igual ao tipo recebido como argumento.

```
double getAverageOccupancy(AidBox aidbox);
```

- Este método deve devolver a média de ocupação de todos os contentores da `AidBox`. A ocupação de cada contentor é calculada através da fórmula: $(\text{últimaMedição} / \text{capacidade}) * 100$. Se um contentor não possuir medições, deve ser ignorado no cálculo da média.

Pergunta 2b (5 valores)

Na classe `ReportImpl`, implemente o método `generate`, gerando um relatório textual.

Regras a considerar:

- O relatório deve percorrer todas as `AidBoxes` devolvidas pelo método `getAidBoxes()` da interface `IInstitution`.
- Para cada `AidBox`, deve ser incluída a informação do código, zona e o número de contentores por cada tipo (utilizando o método `countContainersByType`).
- Apenas devem ser incluídas `AidBoxes` cuja ocupação média (utilizando o método `getAverageOccupancy`) seja superior a 50%.
- O método `generate` deve devolver uma `String` com todo o relatório formatado.

Excertos de Código Fornecidos

```
public class InstitutionImpl implements IInstitution {  
    (...)  
    private AidBox[] aidBoxes;  
    private int numberOfAidBoxes;  
    private Vehicle[] vehicles;  
    private int numberOfVehicles;  
    (...)  
  
    public AidBox[] getAidBoxes() {  
        (...)  
    }  
  
    public Vehicle[] getVehicles() {  
        (...)  
    }  
}
```

```
public interface Route {  
    void addAidBox(AidBox aidBox) throws RouteException;  
    AidBox removeAidBox(AidBox aidBox) throws RouteException;  
    AidBox[] getRoute();  
}
```